Spark Structured Streaming

ADVANCED HANDS-ON EXERCISES

How to use this manual

Work through the labs in order. Use Kafka UI for creating topics, producing JSON messages, and inspecting outputs; CLI alternatives are included. Use the uploaded KRaft Docker Compose environment: kafka (Confluent 7.7.1), kafka-ui (port 8080), and jupyter (port 8888). Spark inside Jupyter connects to kafka:9092; host tools use localhost:29092. Run shell commands on your host unless explicitly marked as notebook code. Each streaming query is given a unique variable name. Stop a query before reusing its sink/checkpoint.

Environment preflight

```
docker compose up -d
docker ps
docker exec -it kafka kafka-topics --bootstrap-server kafka:9092 --list
```

In Jupyter, confirm the Spark Kafka connector is installed by running the Kafka readStream code from the previous manual. The supplied Compose sets PYSPARK_SUBMIT_ARGS to org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3. Confirm Dockerfile.jupyter installs Spark/PySpark 3.5.3 with Scala 2.12 compatibility. If the Kafka source is missing, inspect Jupyter container logs and the package download.

Create the lab topics

```
docker exec kafka kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic orders_json --partitions 2 --replication-factor 1
docker exec kafka kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic fraud_alerts --partitions 1 --replication-factor 1
```

Notebook setup — run once

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType
spark = SparkSession.builder.getOrCreate()
schema = StructType([
    StructField("event_id", StringType()),
    StructField("order_id", StringType()),
    StructField("customer_id", StringType()),
    StructField("product_id", StringType()),
    StructField("amount", DoubleType()),
    StructField("event_time", StringType())
])
raw = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "kafka:9092")
    .option("subscribe", "orders_json")
    .option("startingOffsets", "latest").load())
events = (raw.select(F.from_json(F.col("value").cast("string"), schema).alias("j"))
    .select("j.*")
```

```
    .withColumn("event_ts", F.to_timestamp("event_time")))
print(events.isStreaming)
```

Send sample events

Preferred: open http://localhost:8080 → local-kraft → Topics → orders_json → Produce Message. Paste ONE JSON object into the Value field and produce. Repeat for each message. CLI alternative below: open a host terminal, run the producer, then paste one JSON object per line.

```
docker exec -it kafka kafka-console-producer --bootstrap-server kafka:9092 --topic orders_json
```

```
{"event_id":"e1","order_id":"o1","customer_id":"c1","product_id":"p1","amount":120.0,"event_time":"2026-10-13 15:00:00"}
{"event_id":"e2","order_id":"o2","customer_id":"c2","product_id":"p2","amount":25.0,"event_time":"2026-10-13 15:00:05"}
{"event_id":"e3","order_id":"o3","customer_id":"c1","product_id":"p2","amount":2100.0,"event_time":"2026-10-13 15:00:08"}
```

Important: These sample timestamps are historical. For live window and watermark labs, use timestamps near your actual clock time, or use the supplied Python producer later in the manual. Use fresh event IDs to avoid confusing deduplication tests.

DOCKER COMPOSE + KAFKA UI WORKFLOW

Place docker-compose.yml and Dockerfile.jupyter in the same folder. The uploaded Compose builds the jupyter service from Dockerfile.jupyter; that Dockerfile was not provided, so its installed PySpark version must be verified.

Host terminal: docker compose up -d --build

Host terminal: docker compose ps

Kafka UI: http://localhost:8080 | Jupyter: http://localhost:8888/?token=training123

In Kafka UI: choose local-kraft → Topics → Add a Topic; create orders_json with 2 partitions and fraud_alerts with 1 partition (replication factor 1). If already created using CLI, do not recreate.

To publish: Topics → orders_json → Produce Message → paste a JSON record into Value → Produce Message. To inspect: Topics → orders_json → Messages. Use fraud_alerts → Messages for the capstone.

Networking: inside Jupyter/Spark use kafka:9092; in host Kafka clients use localhost:29092. Do not use localhost:29092 inside Jupyter.

Notebook validation: import pyspark; print(pyspark.__version__) — expect 3.5.3 if the image matches the configured Kafka connector. If different, align the connector with the installed Spark version.

Kafka UI shows broker messages, while Jupyter memory tables show processed records. A successful Kafka produce does not automatically prove the Spark query is active: verify query.isActive and query.lastProgress.

LAB 1 — Parse and validate JSON events

Goal: Convert Kafka bytes to typed records and isolate invalid transactions.

Step 1 — Build valid and invalid streams

Use the common events DataFrame. Reject missing IDs, missing timestamps, and non-positive amounts.

```
valid = events.filter(
    F.col("event_id").isNotNull() &
```

```
    F.col("order_id").isNotNull() &
    F.col("event_ts").isNotNull() &
    (F.col("amount") > 0)
)
invalid = events.filter(
    F.col("event_id").isNull() |
    F.col("order_id").isNull() |
    F.col("event_ts").isNull() |
    F.col("amount").isNull() |
    (F.col("amount") <= 0)
)
```

Step 2 — Start notebook-friendly sinks

Memory sinks make validation visible through spark.sql. Each query reads new Kafka records after it starts.

```
q1_good = (valid.writeStream.format("memory")
    .queryName("valid_orders_lab1").outputMode("append").start())
q1_bad = (invalid.writeStream.format("memory")
    .queryName("invalid_orders_lab1").outputMode("append").start())
# Send NEW producer messages, then:
q1_good.processAllAvailable()
q1_bad.processAllAvailable()
spark.sql("SELECT * FROM valid_orders_lab1").show(truncate=False)
spark.sql("SELECT * FROM invalid_orders_lab1").show(truncate=False)
```

Expected result / validation

Valid JSON orders appear in valid_orders_lab1; malformed JSON or records failing business rules appear in invalid_orders_lab1 (often with null fields).

Your challenge

Use Kafka UI → orders_json → Produce Message to publish a missing order_id and an amount of -10. Confirm both are rejected. Then stop q1_good and q1_bad.

LAB 2 — Real-time order revenue

Goal: Calculate per-customer order count and revenue while new events arrive.

Step 1 — Aggregate

Use a stateful aggregation and complete output mode.

```
revenue = valid.groupBy("customer_id").agg(
    F.count("*").alias("order_count"),
    F.round(F.sum("amount"), 2).alias("revenue")
)
q2 = (revenue.writeStream.format("memory")
    .queryName("customer_revenue_lab2")
    .outputMode("complete").start())
```

Step 2 — Query the result

In Kafka UI, produce at least two NEW events for c1 and one for c2.

```
q2.processAllAvailable()
spark.sql("""
SELECT customer_id, order_count, revenue
```

```
FROM customer_revenue_lab2 ORDER BY revenue DESC
""").show()
```

Expected result / validation

Counts and revenue change as new orders arrive. Complete mode holds the current full aggregation table.

Your challenge

Add an order for a new customer and explain why the aggregation retains state. Stop q2.

LAB 3 — Tumbling versus sliding windows

Goal: Compare fixed 30-second windows with overlapping 30-second windows sliding every 10 seconds.

Step 1 — Create both aggregations

Use event_ts rather than Kafka ingestion timestamp.

```
tumbling = valid.groupBy(F.window("event_ts", "30 seconds")).count()
sliding = valid.groupBy(F.window("event_ts", "30 seconds", "10 seconds")).count()
q3a = (tumbling.writeStream.format("memory")
  .queryName("tumbling_lab3").outputMode("complete").start())
q3b = (sliding.writeStream.format("memory")
  .queryName("sliding_lab3").outputMode("complete").start())
```

Step 2 — Inspect windows

In Kafka UI, publish new events with event_time near the current time, then run:

```
q3a.processAllAvailable()
q3b.processAllAvailable()
spark.sql("SELECT * FROM tumbling_lab3 ORDER BY window.start").show(truncate=False)
spark.sql("SELECT * FROM sliding_lab3 ORDER BY window.start").show(truncate=False)
```

Expected result / validation

A single event belongs to one tumbling window but may contribute to multiple sliding windows.

Your challenge

Send five events 5 seconds apart. Count the windows affected by each event. Stop q3a and q3b.

LAB 4 — Simulate genuinely late events

Goal: Observe event-time watermark behavior using out-of-order event timestamps.

Step 1 — Start watermarked aggregation

Use append mode: finalized windows are emitted only after the watermark passes their end.

```
late_counts = (valid.withWatermark("event_ts", "20 seconds")
  .groupBy(F.window("event_ts", "10 seconds")).count())
q4 = (late_counts.writeStream.format("memory")
  .queryName("late_windows_lab4").outputMode("append")
  .option("checkpointLocation", "/tmp/adv_chk_lab4").start())
```

Step 2 — Drive event time forward

Generate a sequence of events with event_time T, T+10 seconds, T+45 seconds, and then T+5 seconds. Use the producer helper in the appendix or publish hand-crafted JSON with current timestamps. After each group:

```
q4.processAllAvailable()
spark.sql("SELECT * FROM late_windows_lab4 ORDER BY window.start").show(truncate=False)
print(q4.lastProgress)
```

Expected result / validation

After a sufficiently newer event advances the watermark, older windows finalize; an event older than the watermark can be dropped. The watermark is based on maximum observed event time minus delay, not wall-clock waiting.

Your challenge

Compare results when watermark delay changes from 20 to 60 seconds. Stop q4. For a different delay, use a NEW checkpoint path.

LAB 5 — Deduplicate event IDs

Goal: Prevent duplicated Kafka events from being counted more than once.

Step 1 — Create a bounded deduplication stream

Use watermark plus event-time-aware deduplication; the API below requires a Spark version supporting dropDuplicatesWithinWatermark.

```
deduped = (valid.withWatermark("event_ts", "2 minutes")
  .dropDuplicatesWithinWatermark(["event_id"]))
q5 = (deduped.writeStream.format("memory")
  .queryName("dedup_lab5").outputMode("append")
  .option("checkpointLocation", "/tmp/adv_chk_lab5").start())
```

Step 2 — Publish duplicate records

In Kafka UI, publish the same event_id twice, then query:

```
q5.processAllAvailable()
spark.sql("SELECT event_id, order_id FROM dedup_lab5").show()
```

Expected result / validation

Repeated event IDs within the watermark-based deduplication horizon should appear once. State is not retained forever.

Your challenge

Send the same order_id with a different event_id. Explain why it is not removed. Stop q5. If the API is unavailable, use dropDuplicates(["event_id", "event_ts"]) with a watermark and note the different semantics.

LAB 6 — Enrich a stream with static reference data

Goal: Join live order events to product master data.

Step 1 — Create a static product table

This is a stream-static join, not a stream-stream join.

```
products = spark.createDataFrame(
```

```
    [("p1", "Laptop", "Electronics"), ("p2", "Mouse", "Accessories")],
    ["product_id", "product_name", "category"])
enriched = valid.join(F.broadcast(products), "product_id", "left")
```

Step 2 — Start the sink

Use Kafka UI to send orders for p1, p2, and an unknown p9.

```
q6 = (enriched.writeStream.format("memory")
   .queryName("enriched_lab6").outputMode("append").start())
q6.processAllAvailable()
spark.sql("""
SELECT order_id, product_name, category, amount
FROM enriched_lab6
""").show(truncate=False)
```

Expected result / validation

Known products are enriched; unknown products retain the order but have null product metadata due to the left join.

Your challenge

Replace the left join with an inner join. What happens to unknown products? Stop q6.

LAB 7 — Checkpoint recovery and replay

Goal: Verify that a restarted file-sink query resumes from stored Kafka offsets.

Step 1 — Start persistent output

Use a unique directory and checkpoint, not the paths used by earlier labs.

```
q7 = (valid.writeStream.format("parquet")
   .option("path", "/tmp/adv_orders_lab7")
   .option("checkpointLocation", "/tmp/adv_chk_lab7")
   .outputMode("append").start())
# Publish new records, then:
q7.processAllAvailable()
spark.read.parquet("/tmp/adv_orders_lab7").count()
q7.stop()
```

Step 2 — Restart using the same paths

Execute after sending additional events while the query is stopped.

```
q7_restart = (valid.writeStream.format("parquet")
   .option("path", "/tmp/adv_orders_lab7")
   .option("checkpointLocation", "/tmp/adv_chk_lab7")
   .outputMode("append").start())
q7_restart.processAllAvailable()
spark.read.parquet("/tmp/adv_orders_lab7").groupBy("event_id").count().show()
q7_restart.stop()
```

Expected result / validation

New Kafka events are appended after restart without replaying already committed micro-batches. This tests the checkpoint with the same source, sink, and query logic.

Your challenge

Record the number of unique event IDs before and after restart. Explain why deleting the checkpoint changes replay behavior.

LAB 8 — Monitor throughput and Kafka backlog

Goal: Inspect input rate, processed rate, and batch duration; demonstrate a processing bottleneck.

Step 1 — Rate-limit ingestion

Start a query with maxOffsetsPerTrigger, a Kafka source option that limits offsets processed per trigger.

```
limited_raw = (spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", "kafka:9092")
  .option("subscribe", "orders_json")
  .option("startingOffsets", "latest")
  .option("maxOffsetsPerTrigger", 5).load())
q8 = (limited_raw.selectExpr("CAST(value AS STRING) AS value")
  .writeStream.format("memory").queryName("limited_lab8")
  .outputMode("append").trigger(processingTime="5 seconds").start())
```

Step 2 — Inspect metrics

Publish 30+ messages quickly (CLI or Kafka UI) and run:

```
import time
for i in range(4):
  time.sleep(6)
  progress = q8.lastProgress
  if progress:
    print("inputRows:", progress.get("numInputRows"))
    print("inputRate:", progress.get("inputRowsPerSecond"))
    print("processedRate:", progress.get("processedRowsPerSecond"))
    print("duration:", progress.get("durationMs"))
q8.stop()
```

Expected result / validation

With maxOffsetsPerTrigger=5, Spark consumes a bounded number of Kafka records per trigger and may take several batches to catch up. The progress metrics show throughput, not Kafka broker lag directly.

Your challenge

Compare maxOffsetsPerTrigger=5 versus 50. For broker-side lag, inspect consumer offsets with Kafka tools if your setup exposes a stable consumer group.

CAPSTONE — Real-time fraud alert pipeline

Business scenario: ingest order JSON from Kafka, validate it, enrich it with product information, and publish high-value transactions to a fraud_alerts topic. This is a rules-based alert example, not an ML fraud model.

Step 1 — Build the alert stream

```
alerts = (enriched.filter(F.col("amount") >= 1000)
  .select(
    F.col("order_id").cast("string").alias("key"),
    F.to_json(F.struct(
      "event_id", "order_id", "customer_id",
```

```
            "product_name", "amount", "event_time"
        )).alias("value")
    ))
q9 = (alerts.writeStream.format("kafka")
    .option("kafka.bootstrap.servers", "kafka:9092")
    .option("topic", "fraud_alerts")
    .option("checkpointLocation", "/tmp/adv_chk_fraud")
    .start())
```

Step 2 — Read the alerts topic

```
docker exec -it kafka kafka-console-consumer --bootstrap-server kafka:9092 --topic fraud_alerts --from-beginning --property print.key=true --property key.separator=" | "
```

In Kafka UI → orders_json → Produce Message, send a new order with amount 1500 and another with amount 20. In Kafka UI → fraud_alerts → Messages, confirm only the high-value order appears. Only the high-value order should appear in fraud_alerts. Stop the stream with q9.stop(). Kafka sinks can produce duplicates during retries; downstream consumers should deduplicate on event_id or use an idempotent design.

Appendix — Generate current-time JSON events

Run this code in the notebook to generate sample JSON strings with realistic current timestamps. Paste printed lines into the Kafka producer. For watermark tests, adjust seconds_offset to create out-of-order event time.

```
import json, uuid
from datetime import datetime, timedelta
def make_event(seconds_offset=0, amount=100.0, customer="c1", product="p1", event_id=None):
    now = datetime.now() + timedelta(seconds=seconds_offset)
    return json.dumps({
        "event_id": event_id or str(uuid.uuid4()),
        "order_id": str(uuid.uuid4()),
        "customer_id": customer,
        "product_id": product,
        "amount": amount,
        "event_time": now.strftime("%Y-%m-%d %H:%M:%S")
    })
print(make_event(0))
print(make_event(10))
print(make_event(45))
print(make_event(5))
```